

Proposal

Xây dựng hệ thống label-driven search cho knowledge base trong phạm vi source code

1. Bối cảnh

Trong các hệ thống phần mềm có quy mô vừa và lớn, source code thường trải rộng trên nhiều module, nhiều tầng kỹ thuật và nhiều nhóm chức năng khác nhau. Khi sử dụng AI để hỗ trợ tìm kiếm ngữ cảnh, giải thích nghiệp vụ, hỗ trợ debug hoặc trả lời câu hỏi liên quan đến code, một vấn đề thường gặp là phạm vi tìm kiếm quá lớn.

Cách tiếp cận search toàn bộ source code theo kiểu truyền thống thường dẫn đến một số hạn chế:

- lượng dữ liệu quét quá rộng, khó tập trung đúng vùng nghiệp vụ liên quan
- dễ lấy phải nhiều đoạn code nhiễu, làm giảm độ chính xác
- tiêu tốn nhiều token khi đưa ngữ cảnh vào AI
- dễ làm tràn cửa sổ ngữ cảnh, khiến thông tin quan trọng bị lãng
- khó giữ được chất lượng ổn định khi codebase tiếp tục mở rộng

Trong bối cảnh đó, cần một cơ chế giúp AI không phải tìm kiếm trên toàn bộ source code mỗi lần có query, mà có thể thu hẹp phạm vi ngay từ đầu dựa trên tri thức nghiệp vụ đã được chuẩn hóa.

2. Nhu cầu

Mục tiêu của đề xuất này là xây dựng một knowledge base định hướng theo nghiệp vụ cho source code, trong đó mỗi file hoặc đối tượng code được gắn với các label có ý nghĩa nghiệp vụ. Thay vì search toàn cục trên toàn bộ dự án, hệ thống sẽ phân tích query thành các label tương ứng, sau đó chỉ tìm kiếm trong tập code đã được gắn nhãn phù hợp.

Nhu cầu cốt lõi gồm:

- tăng độ chính xác khi tìm kiếm code liên quan đến một nghiệp vụ cụ thể
- giảm phạm vi search để tiết kiệm chi phí xử lý và token
- tăng chất lượng ngữ cảnh cung cấp cho AI
- tạo nền tảng để knowledge base có thể mở rộng và cải thiện theo thời gian
- tận dụng xác nhận từ người có chuyên môn nghiệp vụ để đảm bảo label phản ánh đúng thực tế hệ thống

3. Background và ý tưởng cốt lõi

Ý tưởng trọng tâm là xây dựng một lớp chỉ mục nghiệp vụ nằm trên source code.

Thay vì coi code chỉ là các file kỹ thuật, hệ thống sẽ xem mỗi file, class, function hoặc module là một đối tượng có thể được gắn một hoặc nhiều label nghiệp vụ. Các label này đại diện cho các khái niệm ngắn gọn

nhưng quan trọng trong hệ thống, ví dụ như tên chức năng, quy trình, domain, vai trò dữ liệu, luồng xử lý hoặc business capability.

Nguyên lý hoạt động

1. Xây dựng một tập label chuẩn thường dùng trong hệ thống nghiệp vụ.
2. Duyệt qua toàn bộ source code và gắn label cho các đối tượng code tương ứng.
3. Lưu trữ kết quả gắn nhãn như một lớp metadata phục vụ search.
4. Khi có query, hệ thống phân tích query để ánh xạ về các label phù hợp.
5. Chỉ search trong tập đối tượng code thuộc các label đó thay vì search toàn bộ codebase.
6. Danh sách label được duy trì và cải tiến liên tục thông qua quy trình offline có xác nhận từ chuyên gia nghiệp vụ.

Cách tiếp cận này giúp chuyển bài toán từ "search trên toàn bộ source code" sang "search trên một không gian ngữ nghĩa đã được thu hẹp theo nghiệp vụ".

4. Mô hình đề xuất

4.1. Tầng label nghiệp vụ

Đây là tập label phản ánh các nhóm chức năng hoặc khái niệm quan trọng trong hệ thống. Đặc điểm của label:

- ngắn gọn
- dễ hiểu
- mang tính nghiệp vụ hoặc chức năng
- có thể dùng ổn định trong nhiều truy vấn
- được xác nhận bởi người hiểu domain

Ví dụ, thay vì dựa hoàn toàn vào tên file hoặc tên class, hệ thống có thể dùng các label như:

- thanh toán
- đơn hàng
- phê duyệt
- phân quyền
- đồng bộ dữ liệu
- thông báo
- import/export
- báo cáo
- onboarding
- cấu hình hệ thống

4.2. Tầng mining và quản trị label

Tập label không nên được tạo ngẫu nhiên mà cần có quy trình sinh và kiểm duyệt. Nguồn đầu vào có thể bao gồm:

- tài liệu wiki nội bộ
- tài liệu nghiệp vụ

- tài liệu mô tả module
- tên menu, màn hình, luồng nghiệp vụ
- các từ khóa xuất hiện lặp lại trong dự án

AI có thể hỗ trợ tổng hợp và đề xuất danh sách label ban đầu, sau đó hiển thị trên một bảng để người có chuyên môn xác nhận, chỉnh sửa, gộp hoặc loại bỏ. Đây là quy trình offline, đóng vai trò chuẩn hóa tri thức trước khi đem áp dụng lên source code.

4.3. Tầng gắn nhãn source code

Sau khi có danh sách label chuẩn, hệ thống thực hiện gắn nhãn cho toàn bộ source code. Việc gắn nhãn có thể diễn ra theo nhiều cấp:

- file
- class
- function
- module
- package
- API endpoint
- database query hoặc schema mapping nếu cần

Kết quả gắn nhãn được lưu làm metadata để phục vụ truy xuất nhanh.

4.4. Tầng search theo label

Khi người dùng đặt query, hệ thống không đi search toàn bộ codebase ngay lập tức. Thay vào đó:

- phân tích query
- trích ra các label liên quan
- chọn tập code đã được gắn các label đó
- search sâu hơn trong tập đã rút gọn
- lấy kết quả để cung cấp cho AI

Nhờ đó, AI nhận được ngữ cảnh gọn hơn, đúng trọng tâm hơn và ít nhiễu hơn.

5. Quy trình vận hành đề xuất

Giai đoạn 1: Xây dựng bộ label chuẩn

- thu thập dữ liệu từ wiki, tài liệu nghiệp vụ, mô tả tính năng
- dùng AI để mining và gom nhóm candidate label
- hiển thị danh sách label cho người có chuyên môn xác nhận
- lưu bộ label đã được duyệt

Giai đoạn 2: Đánh nhãn toàn bộ source code

- duyệt qua toàn bộ file trong dự án
- xác định label phù hợp cho từng đối tượng code
- lưu metadata label mapping

- lưu commit tương ứng với trạng thái code đã được gắn nhãn

Giai đoạn 3: Tối ưu cập nhật theo commit

- nếu source code chưa thay đổi commit so với lần trước thì bỏ qua
- nếu có commit mới, so sánh diff với commit đã lưu
- chỉ gắn nhãn lại cho các đối tượng thay đổi
- cập nhật metadata cho phần thay đổi, không xử lý lại toàn bộ

Giai đoạn 4: Tích hợp search cho AI

- query đầu vào được phân tích sang label
- search theo label trước
- chỉ lấy phần code liên quan đưa vào pipeline AI
- AI trả lời dựa trên tập ngữ cảnh đã được thu hẹp

6. Giá trị mang lại

6.1. Tăng độ chính xác

Do chỉ tìm trong vùng code liên quan đến đúng nghiệp vụ, hệ thống có khả năng trả về kết quả sát hơn với ý định của query.

6.2. Giảm nhiễu ngữ cảnh

Khi số lượng file được search giảm xuống, AI ít bị “pha loãng” bởi các đoạn code không liên quan.

6.3. Tiết kiệm token và chi phí

Việc chỉ lấy đúng phần cần thiết giúp giảm lượng context gửi vào model, từ đó giảm chi phí và tăng hiệu quả.

6.4. Phù hợp với codebase lớn

Khi hệ thống mở rộng, mô hình search theo label vẫn giữ được khả năng kiểm soát tốt hơn so với search toàn cục.

6.5. Tăng tính explainable

Vì mỗi kết quả đều đi qua lớp label nghiệp vụ, hệ thống có thể giải thích được vì sao một đoạn code được chọn.

7. Khả năng thực hiện

Đây là ý tưởng có tính khả thi cao vì không đòi hỏi thay đổi trực tiếp cấu trúc source code hiện tại. Hệ thống có thể được xây dựng như một lớp metadata và indexing nằm bên cạnh codebase.

Điều kiện để triển khai được

- có nguồn tài liệu nghiệp vụ hoặc wiki tương đối ổn
- có người hiểu domain để xác nhận label
- có khả năng duyệt source code và trích xuất cấu trúc cơ bản
- có cơ chế lưu metadata theo commit
- có pipeline search hiện tại đủ mở để tích hợp bước filter theo label

Vì sao khả thi

- label là một lớp tri thức bổ sung, không phá vỡ hệ thống hiện tại
- có thể triển khai dần theo module, không cần làm toàn bộ ngay
- có thể bắt đầu từ file-level labeling rồi nâng dần đến function-level
- có thể cập nhật tăng dần theo commit thay vì xử lý lại toàn bộ mỗi lần

8. Hạn chế và rủi ro

8.1. Label có thể thiếu hoặc chưa chuẩn

Ở giai đoạn đầu, tập label chắc chắn chưa bao phủ hết toàn bộ nghiệp vụ.

Hướng xử lý: coi đây là một hệ thống phát triển dần, liên tục mining thêm và điều chỉnh qua phản hồi thực tế.

8.2. Chi phí cập nhật label

Nếu cập nhật toàn bộ source quá thường xuyên sẽ tốn chi phí xử lý.

Hướng xử lý: không cập nhật liên tục theo mọi thay đổi nhỏ; có thể đồng bộ theo chu kỳ release, ví dụ 1 tuần hoặc 2 tuần một lần, hoặc chỉ xử lý incremental theo diff commit.

8.3. Sai lệch giữa label và code thực tế

Một số file có thể bị gắn nhãn chưa đúng, hoặc một file phục vụ nhiều nghiệp vụ.

Hướng xử lý: cho phép nhiều label trên một đối tượng; đồng thời có cơ chế review, đánh giá chất lượng gắn nhãn định kỳ.

8.4. Query không ánh xạ tốt về label

Người dùng có thể dùng từ ngữ khác với bộ label đã định nghĩa.

Hướng xử lý: xây thêm lớp synonym, alias và semantic mapping giữa query và label.

9. Định hướng phát triển tương lai

Sau khi hệ thống nền tảng hoạt động ổn định, có thể mở rộng theo các hướng sau:

9.1. Label đa cấp

Không chỉ là label phẳng, mà tổ chức theo cây phân cấp:

- domain
- sub-domain
- capability
- action
- technical concern

Ví dụ:

- đơn hàng
 - tạo đơn
 - sửa đơn
 - hủy đơn
 - đồng bộ đơn

9.2. Gắn nhãn ở mức sâu hơn

Ban đầu có thể gắn nhãn ở file-level để triển khai nhanh. Về sau nâng lên:

- class-level
- function-level
- API-level
- event / job / cron / queue-level

9.3. Feedback loop từ truy vấn thực tế

Ghi nhận query nào thất bại, query nào phải search ngoài label, query nào cần thêm label mới để cải thiện bộ label theo usage thực tế.

9.4. Kết hợp label search và vector search

Label search không nhất thiết thay thế hoàn toàn cách search cũ. Có thể dùng label như bước filter đầu tiên, sau đó kết hợp semantic retrieval trong vùng đã rút gọn để tăng độ phủ và độ chính xác.

9.5. Đo lường chất lượng

Xây bộ metric để đánh giá:

- precision của search
- recall theo tập use case chuẩn
- token tiết kiệm được
- thời gian phản hồi
- mức độ hài lòng của người dùng nội bộ

9.6. Tự động đề xuất label mới

Khi thấy nhiều query không map tốt vào label hiện tại, hệ thống có thể tự đề xuất label mới để người có chuyên môn duyệt.

10. Kết luận

Đề xuất này hướng tới việc xây dựng một cơ chế knowledge base cho source code dựa trên label nghiệp vụ, nhằm giải quyết bài toán search quá rộng và thiếu chính xác khi cung cấp dữ liệu cho AI.

Ý tưởng cốt lõi không nằm ở việc thay thế hoàn toàn search hiện tại, mà ở việc bổ sung một lớp tri thức giúp thu hẹp đúng vùng code cần quan tâm trước khi đưa vào AI. Nhờ đó, hệ thống có thể:

- tăng độ chính xác
- giảm nhiễu
- tiết kiệm token
- mở rộng tốt hơn theo thời gian
- tận dụng được kiến thức từ người có chuyên môn nghiệp vụ

Dù tập label ban đầu có thể chưa đầy đủ, mô hình này vẫn có khả năng phát triển liên tục thông qua quy trình mining, xác nhận và cập nhật theo chu kỳ. Với cách triển khai incremental theo commit và theo release, đây là một hướng tiếp cận thực tế, khả thi và phù hợp để xây nền tảng knowledge base chuyên sâu cho codebase lớn.